Data Structures and C Programming

Question Paper Answers

S.No	18
Reg No	1972260073B
Name	M Prince Daniel

A. Define Data Structure (5 Marks)

A **data structure** is a systematic way of organizing, storing and managing data in computer memory so that it can be accessed and modified efficiently. It defines the data elements, the relationships among them, and the operations that can be performed on them (insertion, deletion, searching, traversal, sorting).

Classification

Type	Description	Examples
Primitive	Basic types directly supported by the machine	int, float, char, double
Linear	Elements arranged sequentially; each has one predecessor and one successor	Array, Linked List, Stack, Queue
Non-linear	Elements arranged hierarchically or as a network	Tree, Graph

Need / Importance

- Allows efficient storage and fast retrieval of data.
- Reduces time and space complexity of algorithms.
- Helps manage large volumes of data and supports reusability.
- Forms the foundation of operating systems, databases, compilers and networking.

Example: In C, an array `int marks[5];` stores five integers in contiguous memory, accessed by index.

B. C Program for Matrix Multiplication, Addition and Subtraction (10 Marks)

Aim

To write a C program that performs addition, subtraction and multiplication of two matrices using two-dimensional arrays.

Algorithm

- **Addition / Subtraction:** Both matrices must have the same order. Result[i][j] = A[i][j] ± B[i][j].
- **Multiplication:** Columns of A must equal rows of B. Result[i][j] = sum of A[i][k] × B[k][j] for k = 0 to c1–1.
- Read the orders and elements, show a menu, validate dimensions, compute and display the result.

Program

```
#include <stdio.h>

#define MAX 10

void readMatrix(int m[MAX][MAX], int r, int c, char name) {
    printf("Enter elements of matrix %c (%dx%d):\n", name, r, c);
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            scanf("%d", &m[i][j]);
}

void display(int m[MAX][MAX], int r, int c) {
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++)
            printf("%5d", m[i][j]);
        printf("\n");
    }
}

void add(int a[MAX][MAX], int b[MAX][MAX], int res[MAX][MAX], int r, int c) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            res[i][j] = a[i][j] + b[i][j];
}

void subtract(int a[MAX][MAX], int b[MAX][MAX], int res[MAX][MAX], int r, int c) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            res[i][j] = a[i][j] - b[i][j];
}

void multiply(int a[MAX][MAX], int b[MAX][MAX], int res[MAX][MAX],
              int r1, int c1, int c2) {
    for (int i = 0; i < r1; i++)
        for (int j = 0; j < c2; j++) {
            res[i][j] = 0;
            for (int k = 0; k < c1; k++)
                res[i][j] += a[i][k] * b[k][j];
        }
}

int main() {
    int a[MAX][MAX], b[MAX][MAX], res[MAX][MAX];
    int r1, c1, r2, c2, choice;
```

```
    printf("Enter rows and columns of matrix A: ");
    scanf("%d %d", &r1, &c1);
    printf("Enter rows and columns of matrix B: ");
    scanf("%d %d", &r2, &c2);

    readMatrix(a, r1, c1, 'A');
    readMatrix(b, r2, c2, 'B');

    printf("\n1. Addition\n2. Subtraction\n3. Multiplication\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
        case 2:
            if (r1 != r2 || c1 != c2) {
                printf("Operation not possible: dimensions must match.\n");
                return 1;
            }
            if (choice == 1) { add(a, b, res, r1, c1); printf("\nA + B =\n"); }
            else             { subtract(a, b, res, r1, c1); printf("\nA - B =\n"); }
            display(res, r1, c1);
            break;
        case 3:
            if (c1 != r2) {
                printf("Operation not possible: columns of A must equal rows of B.\n");
                return 1;
            }
            multiply(a, b, res, r1, c1, c2);
            printf("\nA x B =\n");
            display(res, r1, c2);
            break;
        default:
            printf("Invalid choice.\n");
    }
    return 0;
}
```

Sample Input and Output

Matrix A = [1 2; 3 4] and Matrix B = [5 6; 7 8]

Operation	Result
Addition (A + B)	[6 8 ; 10 12]
Subtraction (A – B)	[–4 –4 ; –4 –4]
Multiplication (A × B)	[19 22 ; 43 50]

Complexity

- Addition and subtraction: O(r × c) time.
- Multiplication: O(r1 × c1 × c2), which is $O(n^3)$ for square matrices.
- Space: O(r × c) for the result matrix.

C. Role of Arrays in Memory Management (5 Marks)

An **array** is a collection of elements of the same data type stored in **contiguous memory locations**. This layout makes arrays central to how programs use memory.

Key Roles

- **Contiguous allocation:** The compiler reserves one continuous block of size n × sizeof(type), which avoids fragmentation within the array.
- **Fast random access:** Address of element i = base address + i × size of element, so access takes O(1) time.
- **Cache efficiency:** Sequential elements sit next to each other, giving good spatial locality and fewer cache misses.
- **Static allocation:** Fixed-size arrays are allocated at compile time on the stack (or data segment), with no allocation overhead at run time.
- **Dynamic allocation:** Using `malloc()` or `calloc()`, arrays can be created on the heap at run time and released with `free()`.
- **Pointer relationship:** The array name acts as a pointer to its first element, enabling pointer arithmetic and efficient parameter passing.
- **Foundation for other structures:** Stacks, queues, heaps, hash tables and matrices are built on arrays.

Limitations

- Size is fixed for static arrays, so memory may be wasted or insufficient.
- Insertion and deletion in the middle require shifting elements, costing O(n).
- No bounds checking in C, so out-of-range access can corrupt memory.

Example

```
int a[5];                                // static, contiguous block on stack
int *p = (int *)malloc(5 * sizeof(int)); // dynamic, on heap
free(p);                                 // release memory
```